DevOps Interview Preparation Guide

Complete 14-Day Intensive Study Plan

For 4 Years Experience · Senior DevOps / Platform Engineer Level

Target Role: Senior DevOps / Platform Engineer
Experience Level: 4 Years
Study Duration: 14–15 Days (6–8 hours/day)
Core Technologies: Kubernetes · Docker · CI/CD · Terraform · Azure
Total Topics: 150+ Interview Questions & Scenarios

Document Version: March 2026

Day 1 — Kubernetes Core Architecture

Goal: Understand Kubernetes architecture deeply enough to explain how the cluster works internally, draw it on a whiteboard, and debug real production issues. This is the foundation for every other day.

1.1 Why Kubernetes Exists — The Problem It Solves

Before Kubernetes, companies ran applications on Virtual Machines (VMs). A VM includes a full Guest Operating System on top of a Hypervisor, which makes them slow to start, expensive on resources, and difficult to scale quickly.

Docker containers solved packaging problems — you could ship your application and all its dependencies as one image. But containers alone created new problems at scale:

- How do you restart a container automatically when it crashes at 3am?
- How do you distribute thousands of containers across dozens of servers?
- How do you update your application without downtime?
- How do containers on different machines talk to each other?
- How do you manage secrets, configuration, and storage for containers?

Kubernetes solves all of these. It is a container orchestration platform — a system that manages where and how containers run across a cluster of machines, and constantly ensures the cluster is in the state you declared.

> **Interview Answer: What is Kubernetes?**
> Kubernetes is an open-source container orchestration platform that automates deployment, scaling, and management of containerized applications. It maintains a desired state through a continuous reconciliation loop — if the actual state of the cluster drifts from what you declared, Kubernetes automatically corrects it.

1.2 The Kubernetes Cluster — Two Planes

A Kubernetes cluster is divided into two logical parts:

- Control Plane — the brain. Makes decisions about the cluster, stores state, schedules workloads.
- Worker Nodes — the muscles. Where your actual application containers run.

In production (e.g., AKS), you typically never touch the control plane — it is managed by Azure. Worker nodes are VMs (VM Scale Set nodes) that you provision.

1.3 Control Plane Components — Deep Explanation

API Server — The Gateway to Everything

Every single operation in a Kubernetes cluster flows through the API Server — whether it is a kubectl command from your terminal, a CI/CD pipeline deploying a new image, or internal cluster components communicating with each other.

When you run kubectl apply -f deployment.yaml, here is exactly what happens at the API Server:

1. Authentication: Who are you? The request carries a kubeconfig token or certificate.

2. Authorization: Are you allowed to do this? Checked against RBAC rules.
3. Admission Control: Should this request be mutated or rejected? Admission webhooks can modify or block the request.
4. Validation: Is the YAML schema correct? Required fields present?
5. Persistence: Store the object definition in etcd.

Key Interview Point

The API Server is stateless — it does not store anything itself. All state lives in etcd. The API Server is just the interface and validation layer.

etcd — The Cluster Database

etcd is a distributed key-value store that holds the entire state of the cluster. Everything Kubernetes knows about itself lives here — pod definitions, service configurations, secrets, configmaps, RBAC rules, node information.

etcd uses the Raft consensus algorithm, which means it requires a majority of nodes (quorum) to be available before it accepts writes. In production, etcd runs as a cluster of 3 or 5 nodes to ensure high availability.

Critical Interview Point: What happens if etcd fails?

If etcd goes down, the cluster cannot accept any new changes — no new deployments, no scaling, no config updates. However, already running pods continue to run because kubelet on each node works independently. The cluster enters a read-only degraded state until etcd recovers.
etcd is the single source of truth for the entire Kubernetes cluster.

Scheduler — Pod Placement Engine

The Scheduler watches for newly created pods that have not been assigned to a node yet (they have no NodeName set). Its job is to pick the best node for each pod based on a scoring algorithm.

The scheduling process has two phases:

6. Filtering: Remove all nodes that cannot possibly run the pod. Reasons: insufficient CPU, insufficient memory, node tainted and pod has no toleration, node selector labels do not match.
7. Scoring: Among the remaining eligible nodes, score each one and pick the highest score. Scoring considers resource availability, pod affinity/anti-affinity rules, and topology spread constraints.

Interview Scenario: Why is a pod stuck in Pending?

Pending means the Scheduler cannot find a suitable node. Debug with: kubectl describe pod <name> and read the Events section. Common causes:
• Insufficient CPU or memory on all nodes
• Node has a taint that the pod does not tolerate
• nodeSelector labels do not match any node
• PersistentVolumeClaim not bound

Controller Manager — The Reconciliation Engine

The Controller Manager runs a collection of control loops — each loop watches the actual state of some resource and takes action to make it match the desired state. This is the core design principle of Kubernetes: declarative desired state with automatic reconciliation.

Key controllers and what they do:

- ReplicaSet Controller: If you want 3 replicas and only 2 are running, it creates another pod. If 4 are running, it deletes one.
- Deployment Controller: Manages rolling updates by creating new ReplicaSets and scaling them up while scaling old ones down.
- Node Controller: Monitors node heartbeats. If a node stops responding for 5 minutes (default), it marks the node NotReady and begins evicting pods.
- Job Controller: Ensures a Job runs its pods to completion. Handles retries on failure.

Core Kubernetes Philosophy: Reconciliation Loop

Every controller follows the same pattern: Watch the desired state (from etcd). Observe the actual state (from the cluster). Calculate the diff. Take action to close the gap.
This is why Kubernetes is self-healing — the loop runs continuously.

1.4 Worker Node Components

kubelet — The Node Agent

kubelet runs on every single worker node and is responsible for making sure that containers described in PodSpecs are running and healthy. It is the bridge between the control plane and the node.

kubelet's responsibilities:

- Communicates with the API Server to receive pod assignments for its node.
- Instructs the container runtime (containerd) to pull images and start containers.
- Runs health checks (liveness and readiness probes) on containers.
- Reports pod and node status back to the API Server at regular intervals.
- If a container fails its liveness probe, kubelet restarts it automatically.

kube-proxy — Service Networking

kube-proxy runs on every node and is responsible for implementing Kubernetes Service networking. It maintains network rules (using iptables or IPVS) that allow traffic destined for a Service's ClusterIP to be correctly forwarded to one of the healthy backend pods.

Critically, kube-proxy does NOT forward traffic itself — it programs the Linux kernel's networking rules so the OS does the forwarding at line speed.

Mode	How It Works
iptables (default)	Programs iptables DNAT rules. Simpler but can be slow with thousands of services because rules are evaluated sequentially.
IPVS	Uses the kernel's IP Virtual Server module. Hash-table based, O(1) lookup. Used in large clusters with 1000+ services.

Container Runtime — containerd

The container runtime is what actually runs containers. Modern Kubernetes clusters use containerd (the default in AKS and most distributions). Docker was the original runtime but Kubernetes removed direct

Docker support in v1.24 — containerd runs underneath Docker anyway, so images built with Docker work perfectly.

1.5 The Complete kubectl apply Flow

This is one of the most common senior interview questions. Being able to explain this end-to-end, naming each component, demonstrates deep understanding.

Complete Flow: kubectl apply -f deployment.yaml

1. User runs kubectl apply — kubectl reads kubeconfig, forms an HTTP request with the YAML body.
2. API Server receives request — authenticates the user, checks RBAC authorization, runs admission webhooks, validates schema.
3. Stored in etcd — the Deployment object is persisted.
4. Deployment Controller detects new Deployment — creates a ReplicaSet object.
5. ReplicaSet Controller detects new ReplicaSet — creates the required number of Pod objects (in etcd, with no NodeName).
6. Scheduler detects unscheduled pods — filters and scores nodes, writes NodeName to each Pod in etcd.
7. kubelet on the chosen node detects its pod — reads the PodSpec, tells containerd to pull the image.
8. containerd pulls the image from the registry — starts the container.
9. CNI plugin configures networking — assigns a pod IP.
10. kubelet updates pod status — pod transitions to Running.

1.6 Pod Lifecycle — States You Must Know

Phase	Meaning	Common Cause
Pending	Pod accepted, not yet scheduled or image not yet pulled	Insufficient resources, node taint, image pull in progress
Running	At least one container is running	Normal state
Succeeded	All containers exited with code 0	Completed Job or CronJob
Failed	At least one container exited with non-zero code	Application crash, OOMKill, config error
Unknown	Node stopped communicating	Node failure, network partition

1.7 Essential kubectl Commands with Explanation

Do not just memorise these — understand what information each command gives you and when to use it.

```
kubectl get pods -n <namespace>          # List pods and their status
kubectl describe pod <name>              # Full detail: events, conditions, probe
status
kubectl logs <pod> -c <container>        # Container stdout/stderr
kubectl logs <pod> --previous            # Logs from last crashed container
```

```
kubectl exec -it <pod> -- bash           # Shell inside running container
kubectl get events --sort-by=.lastTimestamp  # Cluster events sorted by time
kubectl top pods                         # CPU and memory usage (needs metrics-
server)
kubectl get nodes -o wide                # Node status with IPs and OS
```

1.8 Production Debugging Scenarios

Scenario: Pod stuck in Pending

- Run kubectl describe pod — read the Events section at the bottom.
- Look for messages like: 'Insufficient cpu', 'Insufficient memory', '0/3 nodes are available'.
- If taint-related: '1 node(s) had taint {key:value}, that the pod didn't tolerate'.
- Run kubectl get nodes to see if any nodes are NotReady.
- Check if PVC is bound: kubectl get pvc.

Scenario: Pod in CrashLoopBackOff

- kubectl logs <pod> --previous — view logs from the crashed container instance.
- Common causes: Application exception on startup, missing environment variable, cannot connect to database.
- kubectl describe pod — check if liveness probe is failing and causing restart.
- Check resource limits — if memory limit is too low, container gets OOMKilled.

Scenario: ImagePullBackOff

- kubectl describe pod — Events section shows the exact pull error.
- Causes: Wrong image tag, image doesn't exist, private registry with no imagePullSecret configured.
- Verify: docker pull <image> from a node to test manually.

1.9 Interview Questions with Full Answers

Q: Explain Kubernetes architecture.

Kubernetes has two main parts: the control plane and worker nodes. The control plane consists of the API Server (entry point for all operations), etcd (cluster state database), Scheduler (assigns pods to nodes), and Controller Manager (runs reconciliation loops). Worker nodes run kubelet (node agent), kube-proxy (service networking), and a container runtime like containerd. When you apply a manifest, it flows through the API Server to etcd, controllers create pods, the Scheduler assigns them to nodes, and kubelet starts the containers.

Q: What happens if a node fails?

The Node Controller on the control plane detects the node has stopped sending heartbeats. After a configured timeout (default ~5 minutes), the node is marked NotReady. If the condition persists, pods on that node are evicted and the Scheduler reschedules them on healthy nodes. Pods backed by a Deployment or StatefulSet are automatically recreated. Standalone pods are not rescheduled.

Q: Why is etcd the most critical component?

etcd is the only component that stores cluster state. All other control plane components are stateless and can be restarted. If etcd is lost without backup, the entire cluster state is gone — you lose all your resource definitions, RBAC rules, secrets, and configuration. This is why production clusters run etcd with 3 or 5 replicas across availability zones and take regular etcd snapshots for disaster recovery.

Day 2 — Kubernetes Workloads Deep Dive

Goal: Understand every Kubernetes workload type, how they differ, when to use each, how updates work safely, and how to debug pod-level failures in production.

2.1 Pods — The Atomic Unit

A Pod is the smallest deployable unit in Kubernetes. It is not a container — it is a wrapper around one or more containers that share the same network namespace, IP address, and volume mounts.

Containers inside a pod communicate via localhost because they share a network namespace. This is the sidecar pattern — your main application container and a helper container (logging agent, proxy) run together in one pod.

> **Critical: Pods are ephemeral**
> You should never rely on a pod's IP address or assume it will exist. When a node fails, when you deploy a new version, when autoscaling removes pods — pods are created and destroyed constantly. This is why we never create pods directly in production.

A pod spec includes containers, volumes, environment variables, resource requests/limits, probes, and scheduling constraints. Key pod fields:

- spec.containers[] — the containers to run
- spec.initContainers[] — containers that must complete before main containers start (used for setup tasks like DB migration)
- spec.volumes[] — storage attached to the pod
- spec.nodeSelector / spec.affinity — scheduling constraints
- spec.tolerations — allow scheduling on tainted nodes

2.2 ReplicaSet — Ensuring Pod Count

A ReplicaSet ensures that a specified number of pod replicas are always running. If a pod crashes or is deleted, the ReplicaSet controller creates a replacement. If too many pods are running (e.g., someone manually created extras), it deletes the excess.

ReplicaSets use label selectors to identify which pods they manage. A ReplicaSet with selector app=backend owns all pods with that label.

> **Why not use ReplicaSets directly?**
> Deployments manage ReplicaSets for you and add rolling updates and rollback. In production, you almost always use Deployments, not bare ReplicaSets.

2.3 Deployments — The Standard for Stateless Apps

A Deployment is a higher-level controller that manages ReplicaSets to provide declarative updates for pods. When you update an image tag in a Deployment, it creates a new ReplicaSet with the new image and gradually scales it up while scaling down the old ReplicaSet.

Deployment hierarchy: Deployment → ReplicaSet → Pods

Each time you change a Deployment spec (new image, new env var, new resource limit), Kubernetes creates a new ReplicaSet. Old ReplicaSets are kept (with 0 replicas) for rollback purposes.

2.4 Rolling Updates — How Zero-Downtime Deploys Work

When you update a Deployment, Kubernetes performs a rolling update by default. It does not kill all old pods at once — it gradually replaces them, controlled by two critical parameters:

Parameter	Meaning
maxUnavailable	Maximum number of pods that can be unavailable during the update. Default: 25%. If you have 4 pods, at most 1 can be down at any time.
maxSurge	Maximum number of extra pods that can be created above the desired count during the update. Default: 25%. Allows the cluster to temporarily run more pods during transition.

The update sequence with maxUnavailable=1, maxSurge=1, replicas=3:

8. Kubernetes creates 1 new pod with new image (now 4 pods total — 3 old, 1 new).
9. Waits for the new pod to pass its readiness probe.
10. Terminates 1 old pod (now 3 total — 2 old, 1 new).
11. Repeats until all old pods are replaced.

> **Readiness Probes are Critical for Rolling Updates**
>
> If your new pods do not have readiness probes, Kubernetes will consider them ready immediately and start terminating old pods — even if your new version is broken. Always define readiness probes in production deployments.

2.5 Rollback — Undoing a Bad Deploy

Kubernetes keeps the history of all Deployment revisions. If a new deployment breaks production, you can instantly roll back:

```
kubectl rollout history deployment/my-app     # See revision history
kubectl rollout undo deployment/my-app        # Roll back to previous revision
kubectl rollout undo deployment/my-app --to-revision=3  # Roll back to specific
version
kubectl rollout status deployment/my-app      # Watch rollout progress
```

2.6 Health Probes — The Most Important Production Concept

Kubernetes uses three types of probes to understand the health of your containers. Getting these right is the difference between a stable production cluster and constant firefighting.

Liveness Probe — Is the container alive?

If the liveness probe fails, Kubernetes kills the container and starts a fresh one. Use this for deadlocks — cases where the process is running but stuck and will never recover on its own.

Example: An HTTP server that accepted too many connections and is deadlocked. The process is running (no crash), but it cannot serve requests. A liveness probe hitting /healthz will fail, and Kubernetes restarts it.

Readiness Probe — Is the container ready for traffic?

If the readiness probe fails, Kubernetes removes the pod from the Service's endpoints — it stops receiving traffic. The pod continues running; it is just taken out of rotation. Use this for temporary states: application starting up, loading config, warming cache, upstream dependency temporarily unavailable.

Example: Your Java app takes 30 seconds to start. Without a readiness probe, Kubernetes sends traffic to it before it is ready, causing errors. With a readiness probe, the pod only receives traffic after /ready returns 200.

Startup Probe — For slow-starting containers

The startup probe is checked first when a container starts. While it is failing, liveness and readiness probes are paused. This prevents liveness probes from killing a slow-starting container before it has had a chance to initialize. Once the startup probe succeeds, liveness and readiness probes take over.

Probe Type	When it fails	Use case
Liveness	Container is killed and restarted	Deadlocked processes, unrecoverable errors
Readiness	Pod removed from Service endpoints	Slow startup, temporary unavailability, cache warming
Startup	Liveness/readiness suspended until it passes	Apps needing >30s to initialize (legacy Java, etc.)

2.7 StatefulSets — For Stateful Applications

StatefulSets are used for applications that need stable identity, stable storage, and ordered deployment/scaling. The classic examples are databases (Postgres, MySQL, Cassandra, Kafka, ZooKeeper).

What makes StatefulSets different from Deployments:

- Stable network identity: Pods are named sequentially (pod-0, pod-1, pod-2). The name is stable across restarts. pod-0 always remains pod-0.
- Stable storage: Each pod gets its own PersistentVolumeClaim that is NOT deleted when the pod is deleted. pod-0 always reattaches to its own volume.
- Ordered operations: Pods are created in order (0, 1, 2) and scaled down in reverse order (2, 1, 0). Updates happen one pod at a time, in order.

Why databases need StatefulSets

In a Postgres HA cluster, one pod is the primary (writable) and the others are replicas (read-only). Each pod needs a consistent hostname that other pods can reference for replication. StatefulSet gives pod-0 the hostname postgres-0.postgres-svc.default.svc.cluster.local — stable and predictable.

2.8 DaemonSets — One Pod Per Node

A DaemonSet ensures exactly one pod runs on every node (or a subset of nodes matching a selector). When a new node joins the cluster, the DaemonSet automatically adds a pod to it. When a node is removed, the pod is garbage collected.

Production use cases:

- Log collection agents (Fluentd, Filebeat) — must run on every node to collect node-level logs.
- Monitoring agents (Prometheus node-exporter) — collects node-level metrics.
- Network plugins (CNI agents, kube-proxy).
- Storage drivers (CSI node plugins).

2.9 Jobs and CronJobs — Batch Workloads

A Job runs one or more pods to completion. Unlike a Deployment, it does not restart pods after they succeed — it ensures the task completes successfully.

A CronJob creates Jobs on a schedule (cron syntax). Examples: nightly database backup, hourly report generation, weekly cleanup task.

Workload	Use Case
Deployment	Stateless, long-running applications (web servers, APIs)
StatefulSet	Stateful apps needing stable identity and storage (databases, message queues)
DaemonSet	Node-level agents that must run everywhere (log shippers, monitoring)
Job	One-time batch tasks (data migration, report generation)
CronJob	Scheduled batch tasks (backups, cleanup, scheduled reports)

2.10 Resource Requests and Limits

Every container should define CPU and memory requests and limits. This is critical for cluster stability and is a common interview topic.

Field	What It Controls
requests.cpu	Used by Scheduler for pod placement decisions. Pod is only placed on a node with at least this much free CPU.
requests.memory	Same — Scheduler uses this for placement.
limits.cpu	Container is throttled if it exceeds this. Does NOT cause a restart.
limits.memory	If container exceeds this, it is killed with OOMKilled and restarted.

```
resources:
  requests:
    cpu: '200m'      # 200 millicores = 0.2 CPU cores
    memory: '256Mi'  # 256 megabytes
  limits:
    cpu: '500m'
    memory: '512Mi'
```

OOMKilled Debugging

If you see OOMKilled in kubectl describe pod, the container exceeded its memory limit.
1. kubectl describe pod — check Containers section, 'Last State: OOMKilled'
2. kubectl top pod — see current memory usage
3. Either increase the memory limit or profile your application for memory leaks.

2.11 Taints, Tolerations, Affinity

Taints and Tolerations

A taint is applied to a node to repel pods that do not explicitly tolerate it. This lets you dedicate nodes for specific workloads — GPU nodes for ML workloads, high-memory nodes for databases.

```
# Taint a node (effect: NoSchedule, PreferNoSchedule, NoExecute)
kubectl taint nodes node1 workload=gpu:NoSchedule

# Pod must have a matching toleration
tolerations:
- key: 'workload'
  value: 'gpu'
  effect: 'NoSchedule'
```

Node Affinity

More flexible than nodeSelector. requiredDuringSchedulingIgnoredDuringExecution is a hard rule (pod will not schedule if not met). preferredDuringSchedulingIgnoredDuringExecution is a soft preference.

Pod Anti-Affinity

Ensures replicas of the same application spread across different nodes or availability zones. If all 3 replicas are on the same node and that node fails, your service is down. Anti-affinity prevents this.

Production Pattern: Always use Pod Anti-Affinity

For any production deployment with multiple replicas, add pod anti-affinity to spread replicas across nodes and zones:
topologyKey: kubernetes.io/hostname — spread across different nodes
topologyKey: topology.kubernetes.io/zone — spread across availability zones

2.12 Interview Questions with Full Answers

Q: What is the difference between a Pod and a Deployment?

A Pod is the smallest deployable unit — it runs one or more containers. It has no self-healing; if it dies, it is gone. A Deployment is a higher-level controller that manages a ReplicaSet, which maintains a desired number of pod replicas. Deployments add features like rolling updates, rollback, and self-healing. In production, you always use Deployments for stateless apps.

Q: How does a rolling update ensure zero downtime?

Kubernetes creates new pods with the updated image and waits for them to pass their readiness probe before terminating old pods. The maxUnavailable and maxSurge parameters control how

many pods can be replaced simultaneously. Readiness probes are critical here — they prevent traffic from being sent to pods that are not yet ready, and they act as the signal for Kubernetes to proceed with terminating old pods.

Q: When would you use a StatefulSet instead of a Deployment?

Use a StatefulSet when your application needs stable, unique network identities (consistent pod hostnames), stable persistent storage (each pod's volume survives pod restarts), or ordered deployment and scaling. Databases are the primary use case — Postgres, MySQL, Kafka, Cassandra, and Elasticsearch all typically use StatefulSets.

Day 3 — Kubernetes Networking Deep Dive

Goal: Master Kubernetes networking end-to-end — from how pods get IPs to how traffic flows from the internet through to a pod. Be able to debug any networking issue.

3.1 The Kubernetes Networking Model

Kubernetes networking is built on three fundamental rules that every implementation must satisfy:

12. Every pod gets its own unique IP address. Pods are not NAT'd — they behave like hosts on a flat network.
13. All pods can communicate with all other pods without NAT. A pod on Node A can directly reach a pod on Node B using its pod IP.
14. Nodes can communicate with all pods without NAT.

This is called the flat networking model and it simplifies application design — your app does not need to think about NAT or port mapping. However, implementing this model is the job of the CNI plugin.

3.2 CNI — Container Network Interface

CNI (Container Network Interface) is the standard that Kubernetes uses to plug in different networking implementations. When a pod is created, kubelet calls the configured CNI plugin to: assign an IP to the pod, configure the network namespace, and set up routing so the pod can communicate with other pods and services.

CNI Plugin	Key Characteristics
Azure CNI	Pods get IPs directly from the Azure VNet subnet. Pods are directly routable from VMs outside AKS. Risk: IP exhaustion since each pod consumes a VNet IP.
Kubenet (Azure default)	Pods get IPs from an overlay network (not VNet IPs). Uses NAT for external access. Lower IP consumption but no direct VNet routing.
Calico	Supports NetworkPolicy, BGP routing. Can work with or without overlay. Common in self-managed clusters.
AWS VPC CNI	Pods get VPC IPs. Direct ENI-level routing. Native AWS network integration.

Azure CNI vs Kubenet — When to Use Which

Azure CNI: Use when pods need direct VNet connectivity (e.g., pods talking to Azure PaaS services via private endpoints), or when you need firewall rules at the pod level using NSGs. Downside: you need a large VNet subnet.
Kubenet: Use for dev/test or when IP space is limited. Pods are not directly routable from the VNet without UDR (User Defined Routes).

3.3 Services — Stable Access to Ephemeral Pods

Because pods are ephemeral and their IPs change constantly, you never connect directly to a pod IP. Services provide a stable virtual IP (ClusterIP) and DNS name that routes to the matching pods.

A Service uses a label selector to determine which pods it routes traffic to. It maintains an Endpoints object that lists the current IPs of all matching, healthy pods. kube-proxy programs the networking rules based on this Endpoints list.

ClusterIP — Internal Communication

The default service type. Creates a virtual IP that is only reachable from inside the cluster. Used for internal microservice-to-microservice communication. When the frontend calls backend, it uses the service name, not a pod IP.

```
# Frontend calls backend via service name — DNS resolves this
curl http://backend-service:8080/api/data
# Resolves to: backend-service.default.svc.cluster.local
```

NodePort — External via Node IP

Exposes the service on each node's IP at a static port (30000–32767). Used for development and testing. In production, this is not recommended because it exposes node IPs directly and the port range is limited.

LoadBalancer — Production External Access

Provisions an external load balancer (Azure Load Balancer in AKS, ELB in EKS). External traffic hits the load balancer IP, which routes to NodePort, which routes to the pod. This is how you expose applications to the internet in AKS.

ExternalName — DNS Alias

Maps a service to an external DNS name. Used to give internal cluster services a consistent name that maps to an external database or service.

3.4 kube-proxy — How Service Routing Actually Works

kube-proxy watches the API Server for Service and Endpoints changes. When these change, it updates the iptables (or IPVS) rules on every node so that traffic destined for the Service's ClusterIP gets DNAT'd to one of the pod IPs.

Example iptables flow for a service with ClusterIP 10.0.0.50 and two backend pods:

```
Packet destined for 10.0.0.50:80
  → iptables PREROUTING chain
  → KUBE-SERVICES chain
  → KUBE-SVC-XXXXX (matches service ClusterIP)
  → randomly selects KUBE-SEP-1 (pod 1 at 10.1.0.5:8080) or
                     KUBE-SEP-2 (pod 2 at 10.1.0.6:8080)
  → DNAT rule changes destination to pod IP:port
  → Kernel routes packet to the appropriate node/pod
```

3.5 DNS — Service Discovery Inside the Cluster

Kubernetes runs CoreDNS as a cluster-internal DNS server. Every service gets an automatic DNS A record in the format: <service-name>.<namespace>.svc.cluster.local

Pods are also discoverable via DNS in StatefulSets: <pod-name>.<service-name>.<namespace>.svc.cluster.local

```
# From any pod in the cluster:
# Short form (same namespace):
curl http://backend

# Full form (cross-namespace):
curl http://backend.production.svc.cluster.local

# Debug DNS inside a pod:
kubectl exec -it <pod> -- nslookup backend
kubectl exec -it <pod> -- nslookup backend.production.svc.cluster.local
```

3.6 Ingress — HTTP Routing and TLS Termination

LoadBalancer services are expensive — each one provisions a separate Azure Load Balancer. For HTTP/HTTPS workloads, Ingress provides a single entry point that can route multiple hostnames and URL paths to different backend services.

An Ingress resource defines the routing rules (YAML). An Ingress Controller (e.g., NGINX Ingress, AGIC for Application Gateway) reads these rules and implements them.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /v1
        pathType: Prefix
        backend:
          service:
            name: api-v1-service
            port: { number: 80 }
      - path: /v2
        pathType: Prefix
        backend:
          service:
            name: api-v2-service
            port: { number: 80 }
```

3.7 Network Policies — Zero Trust Pod Networking

By default, all pods can communicate with all other pods in the cluster — this is insecure. Network Policies let you define rules about which pods can communicate with which other pods and on which ports.

Network Policies are enforced by the CNI plugin (Calico, Azure NPM, etc.). You need a CNI that supports NetworkPolicy — not all do.

Example: Deny-all with selective allow

A good production pattern is to deny all ingress traffic by default, then explicitly allow only what is needed:
1. Apply a default-deny NetworkPolicy to a namespace.
2. Allow frontend to backend on port 8080.
3. Allow backend to database on port 5432.
4. Allow monitoring namespace to scrape metrics from all pods on port 9090.
This implements a zero-trust model inside the cluster.

3.8 End-to-End Request Flow — The Full Picture

This is what you should be able to describe fluently in an interview. User accessing your application hosted in AKS via Application Gateway:

15. Client sends HTTPS request to api.example.com.
16. DNS resolves api.example.com to the Application Gateway's public IP.
17. Application Gateway listener receives the request on port 443.
18. WAF rules inspect the request. If clean, routing rule forwards to backend pool.
19. AGIC has mapped the Kubernetes Ingress rules to App Gateway config. Backend pool contains AKS node IPs.
20. Request arrives at AKS node on NodePort.
21. kube-proxy iptables rules DNAT the request to one of the backend pod IPs.
22. Pod processes the request and returns response via the same path.

3.9 Production Networking Debugging

When facing a networking issue, work through layers systematically:

```
# 1. Can the pod reach other pods?
kubectl exec -it <pod> -- curl http://<other-pod-ip>:<port>

# 2. Can the pod reach a service?
kubectl exec -it <pod> -- curl http://<service-name>:<port>

# 3. Is DNS working?
kubectl exec -it <pod> -- nslookup <service-name>

# 4. Check endpoints (are pods registered with the service?)
kubectl get endpoints <service-name>
# If ENDPOINTS is <none>, no pods match the service's label selector

# 5. Check NetworkPolicy
kubectl get networkpolicy -n <namespace>

# 6. Check CoreDNS
kubectl get pods -n kube-system | grep coredns
kubectl logs -n kube-system <coredns-pod>
```

3.10 Interview Questions with Full Answers

Q: Why do we use Services instead of connecting directly to pod IPs?

Pod IPs are ephemeral — they change every time a pod is restarted, replaced during a deployment, or rescheduled. Services provide a stable virtual IP and DNS name that never changes. The Service's Endpoints list is updated dynamically as pods come and go, so clients always connect to the Service and get routed to a healthy pod.

Q: What happens if kubectl get endpoints shows no endpoints for a service?

No endpoints means no pods are matching the service's label selector. Common causes: the pod labels do not exactly match the service selector, all pods are failing their readiness probe so they were removed from the endpoints, or pods exist in a different namespace. Fix by comparing kubectl describe service <name> (check selector) against kubectl get pods --show-labels.

Q: How does a pod on Node A communicate with a pod on Node B?

Via the CNI networking layer. Each pod has a veth (virtual ethernet) pair — one end in the pod's network namespace, one end in the node's namespace. Packets from the pod travel through the veth to the node's network interface. The CNI plugin (e.g., Azure CNI) has set up routing rules so traffic destined for pod IPs on other nodes is routed through the Azure VNet to the correct node, which then delivers it to the target pod's veth. No NAT is involved — it is direct IP routing.

Day 4 — Kubernetes Storage: PVs, PVCs, StorageClasses, ConfigMaps, Secrets

Goal: Understand how Kubernetes handles persistent storage and configuration injection. Be able to design storage for stateful applications and explain the PV/PVC/StorageClass lifecycle.

4.1 Why Storage is Complex in Kubernetes

Containers are ephemeral by design — when a container restarts, all data written to its filesystem is lost. For stateless applications (web servers, APIs), this is fine. For stateful applications (databases, file stores, queues), you need storage that persists beyond the lifetime of any individual pod.

Kubernetes solves this with a decoupled storage model: the infrastructure team provisions storage (PersistentVolumes), and the application teams request it (PersistentVolumeClaims). StorageClasses enable dynamic provisioning so storage is created on-demand.

4.2 PersistentVolume (PV) — The Storage Resource

A PersistentVolume is a piece of storage in the cluster that has been provisioned by an administrator or dynamically created by a StorageClass. It is a cluster-level resource — not namespace-scoped. It represents actual storage: an Azure Disk, an Azure File share, an NFS mount, etc.

A PV has a lifecycle independent of any pod. It persists beyond the pod that uses it. Key PV properties:

- Capacity: How much storage (e.g., 50Gi).
- Access Modes: ReadWriteOnce (mounted by one node), ReadOnlyMany (mounted by many nodes read-only), ReadWriteMany (mounted by many nodes read-write).
- Reclaim Policy: What happens when the PVC is deleted — Retain (keep the data), Delete (delete the underlying storage), Recycle (deprecated).
- StorageClass: Which storage class provisioned this volume.

Access Mode	Description
ReadWriteOnce (RWO)	Can be mounted read-write by a single node. Used for databases (Azure Disk). Most common.
ReadOnlyMany (ROX)	Can be mounted read-only by many nodes simultaneously. Used for shared static content.
ReadWriteMany (RWX)	Can be mounted read-write by many nodes simultaneously. Used for shared file storage (Azure Files, NFS).

4.3 PersistentVolumeClaim (PVC) — The Storage Request

A PVC is a request for storage by a user or application. It is namespace-scoped. The application declares: 'I need 20Gi of storage with ReadWriteOnce access mode.' Kubernetes finds a matching PV and binds them together.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
```

```
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
  storageClassName: managed-premium  # Which StorageClass to use
```

The PVC is then referenced in the pod spec. The pod does not know or care what the underlying storage is — it just mounts the volume at a path.

4.4 StorageClasses — Dynamic Provisioning

Without StorageClasses, an admin has to manually create PersistentVolumes in advance. StorageClasses enable dynamic provisioning — when a PVC is created, Kubernetes automatically provisions the underlying storage (Azure Disk, Azure File) and creates a PV for it.

AKS has several built-in StorageClasses:

StorageClass	Underlying Storage	Use Case
default	Azure Standard HDD	Dev/test, infrequent access
managed-premium	Azure Premium SSD	Production databases
azurefile	Azure Files Standard	Shared file storage (RWX)
azurefile-premium	Azure Files Premium SSD	High-performance shared storage

4.5 StatefulSet + PVC — The Complete Pattern

StatefulSets use volumeClaimTemplates to automatically create a dedicated PVC for each replica. Each pod gets its own volume and always reattaches to its own volume, even after restarts or rescheduling.

> **StatefulSet Storage Pattern**
> If your StatefulSet has 3 replicas named postgres-0, postgres-1, postgres-2, each gets its own PVC:
> data-postgres-0, data-postgres-1, data-postgres-2
> These PVCs are NOT deleted when the StatefulSet is scaled down — data is preserved.

4.6 ConfigMaps — Injecting Configuration

ConfigMaps store non-sensitive configuration data as key-value pairs. They decouple configuration from the container image — the same image can be used in dev, staging, and prod with different ConfigMaps.

Two main ways to use ConfigMaps in pods:

23. As environment variables: each key becomes an env var inside the container.
24. As mounted files: the ConfigMap is mounted as a directory, each key becomes a file. Useful for config files like nginx.conf, application.properties.

```
# ConfigMap as environment variable
env:
- name: DATABASE_HOST
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: database_host

# ConfigMap as mounted file
volumes:
- name: config-vol
  configMap:
    name: nginx-config
volumeMounts:
- name: config-vol
  mountPath: /etc/nginx
```

ConfigMap Limitation

ConfigMaps are NOT encrypted at rest by default. Do not put passwords, tokens, or API keys in ConfigMaps. Use Secrets instead.

4.7 Secrets — Sensitive Configuration

Secrets are similar to ConfigMaps but are designed for sensitive data. They are base64-encoded (not encrypted by default — just encoded to avoid accidental display). For true encryption, use Azure Key Vault integration or enable etcd encryption at rest.

Secret types:

- Opaque: General-purpose key-value pairs. Most common type.
- kubernetes.io/tls: Stores TLS certificates (tls.crt and tls.key).
- kubernetes.io/dockerconfigjson: Registry pull credentials for private images.

Production Best Practice: Use Azure Key Vault

Do not store sensitive secrets as Kubernetes Secrets if you can avoid it. Instead, use the Azure Key Vault Provider for the Secrets Store CSI Driver. This mounts Key Vault secrets directly into pods as files, with automatic rotation. Secrets never touch etcd.

4.8 Interview Questions with Full Answers

Q: What is the relationship between PV, PVC, and StorageClass?

A StorageClass defines HOW storage is provisioned (which Azure disk type, replication, etc.). A PersistentVolumeClaim is an application's request for storage with specific size and access mode requirements. Kubernetes uses the StorageClass to dynamically provision a PersistentVolume that satisfies the PVC. The PV and PVC are then bound together. The pod references the PVC by name — it doesn't know anything about the underlying storage.

Q: What happens to PVC data when a pod is deleted?

The PVC and its data persist. The PV's reclaim policy determines what happens when the PVC itself is deleted — Retain keeps the disk (data preserved, manual cleanup required), Delete

removes the underlying Azure Disk automatically. In StatefulSets, PVCs created by volumeClaimTemplates are never automatically deleted, even when the StatefulSet is deleted. You must manually delete PVCs if you want to remove the data.

Q: What is the difference between ConfigMap and Secret?

Both store configuration data as key-value pairs, but Secrets are intended for sensitive data and are base64-encoded. Kubernetes treats them differently in access control — you can grant RBAC permissions to Secrets separately from ConfigMaps. In practice, Secrets should be backed by external vault solutions (Azure Key Vault) for production clusters, because Kubernetes Secrets are not encrypted at rest by default.

Day 5 — Kubernetes Security: RBAC, Pod Security, Secrets Management

Goal: Master Kubernetes security concepts — RBAC, service accounts, pod security, network policies, and integration with Azure AD. Be able to design a secure cluster and debug access issues.

5.1 Kubernetes Security Layers

Security in Kubernetes operates at multiple layers. A solid interview answer demonstrates you think about all of them:

- Cluster level: API Server authentication, RBAC authorization, admission controllers.
- Node level: Node OS hardening, kubelet security, container runtime security.
- Pod level: Non-root containers, read-only filesystems, capability dropping, securityContext.
- Network level: Network Policies, mTLS between services, egress controls.
- Data level: Secrets encryption at rest, Key Vault integration, image scanning.

5.2 RBAC — Role Based Access Control

RBAC controls who can do what to which Kubernetes resources. It is based on four objects: Role, ClusterRole, RoleBinding, and ClusterRoleBinding.

Object	Description
Role	Defines a set of permissions within a specific namespace. Example: can get, list, and watch pods in the 'production' namespace.
ClusterRole	Like Role but cluster-wide. Can also grant access to non-namespaced resources (nodes, PVs, namespaces themselves).
RoleBinding	Binds a Role or ClusterRole to a user, group, or service account within a specific namespace.
ClusterRoleBinding	Binds a ClusterRole to a subject cluster-wide.

```
# Example: Developer can read pods and logs in 'dev' namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: ['']
  resources: ['pods', 'pods/log']
  verbs: ['get', 'list', 'watch']
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
- kind: User
  name: alice@company.com   # Azure AD user
```

```
roleRef:
  kind: Role
  name: pod-reader
```

5.3 Service Accounts — Pod Identity

A ServiceAccount provides an identity for processes running in a pod. When a pod needs to call the Kubernetes API (e.g., a CI/CD agent deploying to the cluster, or a monitoring tool listing pods), it uses a ServiceAccount token.

Every namespace has a 'default' service account. If a pod does not specify a service account, it uses the default SA. This is dangerous if the default SA has been granted broad permissions.

Best Practice

Create dedicated service accounts for each application that needs cluster API access. Grant only the specific RBAC permissions needed (least privilege). Do not use the default service account for anything sensitive.

5.4 AKS + Azure AD Integration

AKS can be integrated with Azure AD so that kubectl authentication uses Azure AD credentials. Users authenticate with their Azure AD identity and Kubernetes RBAC maps their Azure AD groups to Kubernetes permissions.

This means: a developer who is a member of the 'AKS-Developers' Azure AD group automatically gets the Kubernetes permissions defined in the RoleBinding that references that group — no separate Kubernetes user management.

5.5 Pod Security — securityContext

The securityContext field in a pod spec controls the security configuration of containers. Key settings:

```
securityContext:
  runAsNonRoot: true          # Container cannot run as root (UID 0)
  runAsUser: 1000             # Run as this specific UID
  readOnlyRootFilesystem: true # Container filesystem is read-only
  allowPrivilegeEscalation: false  # Cannot gain more privileges than parent
  capabilities:
    drop: ['ALL']             # Drop all Linux capabilities
    add: ['NET_BIND_SERVICE'] # Add back only what is needed
```

Why run as non-root?

If an attacker exploits a vulnerability in your application running in a container and gains code execution, running as root gives them root privileges inside the container. If they then escape the container (via a container runtime vulnerability), they have root on the node. Running as non-root limits the blast radius significantly.

5.6 Secrets Management Best Practices

Kubernetes Secrets are base64-encoded, not encrypted. They are stored in etcd in plaintext unless you specifically enable etcd encryption. This means anyone with etcd access can read all your secrets.

Production-grade secrets management:

25. Use Azure Key Vault with the CSI Secrets Store Driver. Secrets are fetched from Key Vault at pod startup and mounted as files or env vars. They never touch etcd.
26. Enable etcd encryption at rest in your AKS cluster.
27. Use Managed Identity to authenticate to Key Vault — no static credentials.
28. Enable audit logging for secret access.
29. Never put secrets in ConfigMaps or application code.

5.7 Network Policies — Zero Trust Networking

Without Network Policies, all pods in a cluster can communicate freely. A compromised pod can reach any other pod, any database, any service. Network Policies implement micro-segmentation.

```
# Deny all ingress to backend namespace by default
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: backend
spec:
  podSelector: {}   # Applies to all pods in namespace
  policyTypes: [Ingress]
  # No ingress rules = deny all ingress
---
# Allow only frontend pods to reach backend pods
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
  namespace: backend
spec:
  podSelector:
    matchLabels: {app: backend}
  ingress:
  - from:
    - namespaceSelector:
        matchLabels: {name: frontend}
    ports:
    - port: 8080
```

5.8 Interview Questions with Full Answers

Q: How do you debug an RBAC access denied error?

Use kubectl auth can-i to test permissions: 'kubectl auth can-i get pods --namespace production --as alice@company.com'. Then use kubectl describe rolebinding and kubectl describe clusterrolebinding to see what permissions are granted. Check that the user/group is correctly mapped in the binding. Remember that RBAC is additive — you cannot deny specific actions, only grant them. If the user has cluster-admin, they have access to everything.

Q: How does a pod securely access Azure Key Vault?

Using Workload Identity (the modern approach for AKS). The pod is associated with a Kubernetes ServiceAccount, which is federated with an Azure Managed Identity via OIDC. The Azure Managed Identity has Key Vault Secrets User RBAC role on the Key Vault. When the pod starts, it gets a token from Azure AD and uses it to authenticate to Key Vault — no static credentials anywhere. The CSI Secrets Store Driver mounts the secrets as files in the pod.

Days 6 & 7 — CI/CD Pipelines: Design, Implementation, and Debugging

Goal: Design production-grade CI/CD pipelines, explain the build/deploy separation, handle multi-repo and multi-environment scenarios, and debug pipeline failures.

6.1 What CI/CD Actually Means

Continuous Integration (CI) is the practice of automatically building and testing code every time a developer commits. The goal is to catch integration problems early — before they compound.

Continuous Delivery/Deployment (CD) is the practice of automatically delivering tested code to environments. Continuous Delivery means the code is always in a deployable state (deploy is triggered manually). Continuous Deployment means every successful build automatically deploys to production.

> **The Golden Rule of CI/CD**
> Build once, deploy many times. Create a single artifact (Docker image tagged with git commit SHA) in CI, then promote that same artifact across Dev → QA → Staging → Prod. Never rebuild for each environment — rebuilding introduces risk that different environments run different code.

6.2 Pipeline Architecture — Stages

A well-designed CI/CD pipeline has clearly separated stages:

30. Source: Trigger on push, PR, or schedule. Checkout code.
31. Build: Compile code, run static analysis (SonarQube, ESLint). Fail fast on errors.
32. Test: Unit tests, integration tests. Code coverage checks. Fast feedback.
33. Package: Build Docker image. Tag with commit SHA and semantic version.
34. Security Scan: Container image scanning (Trivy, Snyk). Dependency vulnerability check.
35. Push: Push image to registry (ACR, ECR). Keep images immutable.
36. Deploy to Dev: Automated. Triggered on every merge to develop branch.
37. Deploy to Staging: Automated or semi-automated. Triggered on merge to main.
38. Deploy to Prod: Manual approval gate. Deploy same image as staging.

6.3 Docker Image Tagging Strategy

Every Docker image must have a meaningful, unique tag. Never use 'latest' in production — it is mutable and you lose traceability.

Tag Strategy	Example
Git commit SHA	myapp:a3f8c2d — always unique, fully traceable
Semantic version	myapp:v1.4.2 — human-readable, good for releases
Branch + commit	myapp:main-a3f8c2d — includes branch context
Latest (avoid in prod)	myapp:latest — mutable, dangerous, loses traceability

6.4 Multi-Repo CI/CD — Interview Scenario

You have two repositories: Repo-X (application code) and Repo-Y (infrastructure scripts). Each needs its own pipeline. This is a common interview scenario.

Best practice: Separate pipelines per repository. Each pipeline has its own triggers, its own scripts, and its own deployment targets.

```
# Pipeline for Repo-X (GitHub Actions example)
on:
  push:
    branches: [main]
    paths: ['src/**', 'Dockerfile']  # Only trigger on relevant changes

jobs:
  build-and-deploy:
    steps:
    - uses: actions/checkout@v3
    - name: Build Docker image
      run: docker build -t myapp:${{ github.sha }} .
    - name: Push to ACR
      run: docker push myregistry.azurecr.io/myapp:${{ github.sha }}
    - name: Deploy to AKS
      run: kubectl set image deployment/myapp
myapp=myregistry.azurecr.io/myapp:${{ github.sha }}
```

6.5 Environment Promotion Strategy

The same Docker image moves through environments. Configuration differences (database URLs, feature flags) are injected via environment-specific ConfigMaps and Secrets, not baked into the image.

39. Merge to develop → auto deploy to Dev with dev ConfigMap.
40. Merge to main → auto deploy to Staging with staging ConfigMap.
41. Tag release → manual approval → deploy same image to Prod with prod ConfigMap.

> **Never Rebuild Between Environments**
>
> If you rebuild the Docker image for production, you cannot guarantee it is identical to what was tested in staging. Always use the same image tag throughout the promotion pipeline. The SHA tag makes this verifiable.

6.6 Secrets in Pipelines

Never store secrets (passwords, API keys, tokens) in your pipeline YAML or in your repository. Use the secret management features of your CI/CD platform:

- GitHub Actions: GitHub Secrets — encrypted, masked in logs, accessible via ${{ secrets.MY_SECRET }}.
- Azure DevOps: Library variable groups, linked to Azure Key Vault. Pipeline can access secrets without seeing their values.
- Jenkins: Jenkins Credentials Manager. Access via credentials() binding.

6.7 Blue-Green and Canary Deployments

Blue-Green

Two identical environments: Blue (current production) and Green (new version). Deploy the new version to Green, run smoke tests, then switch the load balancer from Blue to Green. Instant cutover with instant rollback capability — just switch back to Blue.

Canary

Route a small percentage of traffic (5%, 10%) to the new version first. Monitor error rates and latency. Gradually increase traffic to the new version. If metrics degrade, route all traffic back to the old version. This approach validates the new version with real production traffic before full rollout.

6.8 CI/CD Tools Comparison

Tool	Strengths	Best For
Jenkins	Maximum flexibility, huge plugin ecosystem, can do anything	Complex on-premise pipelines, legacy environments
GitHub Actions	Tight GitHub integration, marketplace, simple syntax	GitHub-hosted projects, modern cloud-native
Azure DevOps	Deep Azure integration, enterprise features, YAML + classic editor	Microsoft stack, AKS, Azure deployments
GitLab CI	Built-in DevSecOps, Auto DevOps, registry, environments	GitLab-hosted projects, complete DevOps platform

6.9 Common Pipeline Failures and Debugging

Failure	Debug Approach
Build fails: cannot find package	Check dependency versions, cache invalidation, network access to package registry
Docker build fails	Run docker build locally. Check Dockerfile syntax, base image availability, file COPY paths
Tests pass locally but fail in pipeline	Environment differences: OS, Node version, env vars, file paths, timezone
Image push fails: unauthorized	Check ACR credentials, service principal expiry, RBAC permissions on registry
Deploy to AKS fails: context error	Check kubeconfig setup in pipeline, AKS credentials, RBAC on AKS cluster
Rolling update stalls	kubectl rollout status — check if new pods are passing readiness probes
Wrong env vars in production	Check that the pipeline is injecting the correct env-specific configmap/secret

6.10 Interview Questions with Full Answers

Q: How do you design a CI/CD pipeline for 10 microservices?

Each microservice has its own pipeline triggered by changes to its own repository or directory. Each pipeline follows the same pattern: build, test, scan, push image to ACR with commit SHA tag,

deploy to dev. A separate release pipeline handles promotion to staging and production with manual approval gates. Shared pipeline logic (common build steps, deploy scripts) lives in a shared library or template repository. This gives independent deployment velocity — one team can release without affecting others.

Q: What is the difference between Continuous Delivery and Continuous Deployment?

Continuous Delivery means every commit is built, tested, and made deployable — but the actual production deployment requires a manual trigger or approval. Continuous Deployment goes one step further: every successful build automatically deploys to production. Most enterprises use Continuous Delivery with a manual approval gate for production, because they need human sign-off for compliance, change management, or risk reasons.

Day 8 — Docker Deep Dive

Goal: Master Docker from image building to networking to multi-stage builds. Know every layer of how containers work and be able to write production-grade Dockerfiles.

8.1 Containers vs VMs — The Fundamental Difference

Virtual Machines run a full guest OS (kernel included) on top of a hypervisor. This provides strong isolation but at the cost of gigabytes of disk, gigabytes of RAM, and minutes to boot.

Containers share the host OS kernel. They use Linux kernel features — namespaces and cgroups — to create isolated environments. This makes containers lightweight (MBs, not GBs), fast to start (milliseconds), and dense (you can run hundreds on one host).

Aspect	VMs vs Containers
Isolation	VM: Full hardware virtualization. Container: Process-level isolation via namespaces.
Startup time	VM: Minutes (boot entire OS). Container: Milliseconds (start process).
Size	VM: Gigabytes (full OS). Container: Megabytes (app + dependencies only).
Density	VM: Dozens per host. Container: Hundreds per host.
Security boundary	VM: Stronger (separate kernel). Container: Weaker (shared kernel).

8.2 Docker Architecture

Docker has a client-server architecture:

- Docker CLI: The command-line tool you interact with (docker build, docker run, docker push).
- Docker Daemon (dockerd): The background service that manages containers, images, networks, volumes.
- containerd: The low-level container runtime that dockerd delegates to. Kubernetes also uses containerd directly.
- runc: The OCI-compliant runtime that actually creates and runs containers using Linux kernel primitives.

8.3 Docker Images — Layers

A Docker image is a stack of read-only layers. Each instruction in a Dockerfile creates a new layer. When you run a container, Docker adds a thin read-write layer on top.

Layers are cached. If a layer's instruction has not changed, Docker reuses the cached layer instead of rebuilding it. This makes subsequent builds fast. Order matters — put frequently changing instructions (like COPY . .) near the bottom, and stable instructions (like RUN apt-get install) near the top.

```
# Bad: COPY happens before dependencies, invalidates cache on every code change
FROM node:18
COPY . .
RUN npm install

# Good: Dependencies cached separately from application code
```

```
FROM node:18
WORKDIR /app
COPY package.json package-lock.json ./   # Only copy dependency files
RUN npm ci --only=production              # Install deps (cached until package.json
changes)
COPY . .                                  # Copy code last (changes frequently)
CMD ['node', 'server.js']
```

8.4 Multi-Stage Builds — Production Best Practice

Multi-stage builds use multiple FROM statements in one Dockerfile. Each stage has a purpose. You copy only the artifacts needed for the final image from build stages. The result is a small, secure production image.

```
# Stage 1: Build
FROM node:18 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build        # Generates /app/dist

# Stage 2: Production runtime
FROM node:18-alpine AS production
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER node                # Run as non-root
EXPOSE 3000
CMD ['node', 'dist/server.js']

# Build stage has compilers, dev tools, test files — NOT in final image
# Final image: only runtime + compiled artifacts = smaller, fewer vulnerabilities
```

Why Multi-Stage Builds Matter

A Java application built with Maven in a single stage includes Maven, JDK, source files, test files — hundreds of MBs. With multi-stage: Stage 1 uses a full JDK to compile, Stage 2 uses only JRE to run. The final image is 3-5x smaller, with a significantly reduced attack surface.

8.5 Dockerfile Best Practices

- Use specific base image tags (node:18.17.0-alpine3.18), not node:latest — prevents unexpected version changes.
- Use alpine or distroless base images for production — minimal attack surface.
- Run as non-root user — add USER directive.
- Use .dockerignore to exclude node_modules, .git, test files from build context.
- One process per container — do not run multiple services in one container.
- Use COPY instead of ADD unless you specifically need ADD's tar extraction or URL features.
- Use ENV for configuration that may change, ARG for build-time variables.

8.6 Container Networking

Docker creates virtual networks for containers:

- Bridge (default): Containers on the same bridge network can communicate by name. Used for docker-compose setups.
- Host: Container shares the host's network namespace directly. Maximum performance, no isolation.
- None: No network access — fully isolated.
- Overlay: For multi-host networking (Docker Swarm). Not commonly used now that Kubernetes handles this.

8.7 Container Storage

- Volume: Managed by Docker, stored in /var/lib/docker/volumes/. Preferred for persistent data.
- Bind mount: Maps a host directory directly into the container. Used for development (live reload).
- tmpfs: Stored in host memory only. Fast, but not persistent. Used for temporary sensitive data.

8.8 Interview Questions with Full Answers

Q: What is the difference between CMD and ENTRYPOINT?

ENTRYPOINT defines the main executable of the container — it is not overridable without --entrypoint flag. CMD provides default arguments to ENTRYPOINT, or the default command if ENTRYPOINT is not set. Best practice: Use ENTRYPOINT for the main process (e.g., the application binary) and CMD for default arguments. Example: ENTRYPOINT ['python', 'app.py'] and CMD ['--port=8080'] — you can override the port by passing a different argument at runtime.

Q: How do you reduce Docker image size?

Use multi-stage builds to exclude build tools from the final image. Use alpine or distroless as base images. Combine RUN commands with && to minimize layers. Use .dockerignore to exclude unnecessary files from the build context. Remove package caches in the same RUN instruction that installs packages (rm -rf /var/cache/apk/*). Use specific base image tags to avoid pulling unnecessary variants.

Day 9 — Terraform & Infrastructure as Code

Goal: Master Terraform state management, module design, environment isolation, and be able to design IaC for a complete Azure production environment.

9.1 Why Infrastructure as Code

Infrastructure as Code (IaC) means defining your infrastructure (VNets, VMs, AKS clusters, databases) in configuration files that are version-controlled, reviewed, and applied automatically. Benefits:

- Version control: Every infrastructure change is a Git commit. Full audit trail.
- Repeatability: Same code creates identical infrastructure in dev and prod.
- Code review: Infrastructure changes go through PR review, just like application code.
- Disaster recovery: Recreate the entire environment from code after a failure.
- Team collaboration: Multiple engineers can work on infrastructure safely.

9.2 Terraform Core Concepts

Concept	What It Is
Provider	Plugin that talks to a cloud API (azurerm for Azure, aws for AWS). Translates Terraform resources to API calls.
Resource	A single piece of infrastructure (azurerm_kubernetes_cluster, azurerm_virtual_network).
State	A file (terraform.tfstate) mapping Terraform resources to real infrastructure. Terraform consults state to determine what to create, update, or delete.
Plan	terraform plan shows what changes will be made without making them. Always review.
Apply	terraform apply executes the plan and creates/modifies/destroys infrastructure.
Module	A reusable, parameterised collection of resources. Think of it as a function in code.
Data Source	Read-only reference to existing infrastructure not managed by this Terraform config.

9.3 Terraform State — The Most Critical Concept

The state file is what allows Terraform to know what currently exists. Without state, Terraform cannot determine if a resource needs to be created, updated, or is already in the desired state.

Problems with local state in team environments:

- State files cannot be shared easily between team members.
- Two people running terraform apply simultaneously can corrupt the state.
- State file in git repository exposes sensitive data (passwords, connection strings stored in state).

Remote State with Azure Storage

```
terraform {
  backend 'azurerm' {
    resource_group_name  = 'terraform-state-rg'
    storage_account_name = 'tfstate12345'
    container_name       = 'tfstate'
    key                  = 'prod/aks.tfstate'  # Different key per environment
  }
}
```

State locking: Azure Blob Storage uses blob leases to lock the state file when terraform apply is running. If another engineer runs apply simultaneously, they get a 'state locked' error and must wait. This prevents concurrent modifications and state corruption.

9.4 Terraform Modules — Reusable Infrastructure

A module is a directory of .tf files that can be called from other Terraform configurations with input variables. Modules allow you to define standard infrastructure patterns once and reuse them across projects, environments, and teams.

```
# Calling an AKS module
module 'aks' {
  source = './modules/aks'

  cluster_name        = 'prod-aks'
  resource_group_name = azurerm_resource_group.main.name
  node_count          = 3
  node_vm_size        = 'Standard_D4s_v3'
  kubernetes_version  = '1.27.3'
  vnet_subnet_id      = module.network.aks_subnet_id
}

# The module handles all the complexity internally:
# - system node pool
# - managed identity
# - RBAC integration
# - monitoring configuration
```

Module Best Practices

Keep modules small and focused — one module per major component (AKS, networking, Key Vault).
Use variables for everything that changes between environments (region, SKU, node count).
Use outputs to expose values other modules need (subnet IDs, cluster name, managed identity ID).
Version your modules — use Git tags and reference them with ?ref=v1.2.0.

9.5 Environment Isolation Strategies

Different environments (dev, staging, prod) should have separate state files. Three common approaches:

Approach	Description
Separate directories	environments/dev/, environments/staging/, environments/prod/ each with their own terraform.tfstate. Clean separation, independent apply. Most recommended.
Workspaces	terraform workspace new prod. Single codebase, multiple state files. Useful for identical environments but variables are harder to manage.
Terragrunt	A wrapper around Terraform that adds DRY configurations and remote state management. Good for large-scale multi-environment setups.

9.6 Common Terraform Scenarios and Solutions

Scenario	Solution
Two engineers apply simultaneously	Remote state with blob lease locking. Second engineer gets 'locked' error.
Resource exists outside Terraform	terraform import azurerm_resource_group.main /subscriptions/.../resourceGroups/my-rg
State file corrupted/lost	Restore from Azure Storage backup. Enable versioning on the storage account.
Need to destroy one resource without touching others	terraform destroy -target=azurerm_kubernetes_cluster.main
Config drift: manual change made to Azure portal	terraform plan shows the drift as a change. terraform apply corrects it.
Sensitive values in state	Use Key Vault references. Encrypt state storage. Restrict access to storage account.

9.7 Full AKS + App Gateway Design with Terraform

A real production Terraform structure for AKS on Azure:

```
infrastructure/
  modules/
    network/          # VNet, subnets, NSGs
    aks/              # AKS cluster, node pools, managed identity
    app-gateway/      # Application Gateway, WAF policy
    key-vault/        # Key Vault, access policies
    monitoring/       # Log Analytics, diagnostics
  environments/
    dev/
      main.tf         # Calls modules with dev-specific variables
      variables.tf
      terraform.tfvars
      backend.tf      # Points to dev state file
    prod/
      main.tf         # Same modules, prod-specific variables
      terraform.tfvars
      backend.tf      # Points to prod state file
```

9.8 Interview Questions with Full Answers

Q: What is Terraform state and why is it important?

Terraform state is a JSON file that maps Terraform resource definitions to real infrastructure in the cloud. It records the unique IDs of every resource Terraform has created. Without state, Terraform cannot determine what exists versus what needs to be created. It also stores metadata like resource dependencies and outputs. Remote state in Azure Blob Storage allows team collaboration and state locking to prevent concurrent apply conflicts.

Q: How do you handle secrets in Terraform?

Never hardcode secrets in .tf files or commit them to git. Use variables marked as sensitive: true so they are not shown in plan output. In CI/CD, inject secrets via environment variables (TF_VAR_db_password). For Azure, use the Key Vault data source to read secrets from Key Vault during apply rather than passing them as variables. The Terraform state file can contain sensitive values — ensure the storage account has access controls, versioning, and encryption enabled.

Q: What happens when you run terraform plan?

Terraform reads all .tf files, reads the remote state, queries the cloud provider API to get the current state of all resources, and computes a diff. It shows: resources to create (green +), resources to change (yellow ~), and resources to destroy (red -). The plan is a preview only — nothing is changed. Always review the plan carefully before apply, especially in production. Look for unexpected destroys — these often indicate a resource replacement that will cause downtime.

Days 10 & 11 — Azure Architecture, Networking & Monitoring

Goal: Design production Azure architectures using AKS, Application Gateway, Private Endpoints, Key Vault, and NAT Gateway. Master monitoring with Prometheus and Grafana.

10.1 Production Azure Architecture — The Full Picture

A production Azure architecture for a microservices application typically involves these layers working together:

42. DNS: Azure DNS resolves the application domain to the Application Gateway's public IP.
43. Application Gateway (with WAF): Single entry point for all HTTP/HTTPS traffic. Handles TLS termination, WAF protection, and layer 7 routing.
44. AKS Cluster: Hosts all application microservices as pods across multiple node pools.
45. Private Endpoints: Secure connectivity from AKS pods to Azure PaaS services (Storage, SQL, Redis) without public exposure.
46. Key Vault: Stores secrets, certificates, and encryption keys. Accessed via Managed Identity.
47. NAT Gateway: Provides a fixed outbound IP for AKS pods reaching external services.
48. Azure Monitor / Prometheus: Captures metrics, logs, and alerts.

10.2 Application Gateway with AGIC

In AKS, the Application Gateway Ingress Controller (AGIC) bridges Kubernetes and Azure Application Gateway. AGIC is a pod running inside the AKS cluster that watches Kubernetes Ingress resources and automatically translates them into Application Gateway configuration.

When you create a Kubernetes Ingress rule, AGIC:

49. Detects the new Ingress resource via the Kubernetes API Server.
50. Parses the routing rules (host, path, backend service).
51. Creates the corresponding Application Gateway configuration: listeners, routing rules, backend pools, health probes.
52. Pushes the configuration to Application Gateway via Azure Resource Manager APIs.

AGIC Common Failure: Permission Issues

AGIC needs a Managed Identity with Contributor rights on the Application Gateway resource. If AGIC is not syncing Ingress rules to the gateway, check:
1. kubectl logs -n kube-system <agic-pod> — look for authentication errors
2. Verify Managed Identity has correct role assignment
3. Check that AGIC has network access to Azure ARM APIs

10.3 Private Endpoints — Deep Understanding

A Private Endpoint creates a network interface inside your VNet with a private IP address that maps to an Azure PaaS service. Traffic to that service travels entirely within the Azure backbone — it never goes to the public internet.

The Private DNS Zone is the critical piece. When you create a private endpoint for a storage account, Azure creates a private DNS zone (privatelink.blob.core.windows.net) and adds an A record mapping the storage account's FQDN to the private endpoint IP. You must link this DNS zone to your VNet, otherwise DNS resolution returns the public IP.

```
# Debug: Is DNS resolving to private IP or public IP?
# From inside the VNet (AKS pod or VM):
nslookup mystorageaccount.blob.core.windows.net

# Expected: returns 10.x.x.x (private IP from your subnet)
# If returning 52.x.x.x (public IP): Private DNS Zone not linked to VNet

# Fix: Link Private DNS Zone to VNet in Azure Portal or Terraform
```

10.4 NAT Gateway — Outbound Traffic Management

By default, AKS nodes and pods use Azure SNAT for outbound internet connectivity. SNAT is shared among all nodes and can suffer from port exhaustion under high connection loads — causing intermittent connection failures.

NAT Gateway solves this by providing a dedicated, static outbound IP address for all traffic leaving the AKS subnet. Benefits:

- Fixed outbound IP: Can be whitelisted by external services (APIs, partners).
- High port count: NAT Gateway provides up to 64,512 SNAT ports per IP address.
- No SNAT port exhaustion under normal conditions.

10.5 TLS/SSL — Complete Deep Dive

TLS (Transport Layer Security) provides three things: encryption (data cannot be read in transit), integrity (data cannot be tampered with), and authentication (you are talking to the real server, not an impersonator).

TLS Handshake — Step by Step

53. Client Hello: Client sends supported TLS versions, cipher suites, and a random nonce.
54. Server Hello: Server selects TLS version and cipher suite, sends its certificate (contains public key and CA signature).
55. Certificate Verification: Client verifies the certificate: is it signed by a trusted CA? Is the domain in the Subject/SAN matching? Is it expired?
56. Key Exchange: Client generates a pre-master secret, encrypts it with the server's public key, sends it. Only the server can decrypt this with its private key.
57. Session Keys Derived: Both sides derive the same symmetric session key from the pre-master secret. Asymmetric crypto ends here.
58. Encrypted Channel: All data is now encrypted with the symmetric session key. Symmetric encryption is used because it is much faster than asymmetric.

TLS Issue	Root Cause and Fix
Certificate expired	Renew certificate. In production, automate renewal with cert-manager.

TLS Issue	Root Cause and Fix
CN/SAN mismatch	Browser checks certificate SAN against the hostname. If api.example.com is not in SAN, TLS fails. Fix: reissue cert with correct SANs.
Incomplete certificate chain	Server must send full chain: Server Cert + Intermediate CA + (optionally Root CA). Missing intermediate causes failures on some clients.
TLS version mismatch	Client requires TLS 1.3, server configured for TLS 1.2 max. Align configuration.
502 from App Gateway	Backend health probe failing, readiness probe failure, backend port mismatch. Check pod status and App Gateway backend health.

```
# Debug TLS from command line
# Check certificate details, chain, and expiry:
openssl s_client -connect api.example.com:443 -servername api.example.com

# From inside a pod:
curl -v https://backend-service.internal.example.com

# Check certificate expiry:
echo | openssl s_client -connect api.example.com:443 2>/dev/null | openssl x509 -
noout -dates
```

10.6 IAM — Managed Identity vs Service Principal

Feature	Managed Identity vs Service Principal
Credential management	Managed Identity: None required — Azure manages the token lifecycle automatically. Service Principal: Client ID + Secret or Certificate that you manage and rotate.
Secret expiry	Managed Identity: No secrets, no expiry. Service Principal: Secrets expire (max 2 years for AAD), causing outages if not rotated.
Use case	Managed Identity: Azure resources authenticating to Azure services (AKS → Key Vault). Service Principal: Non-Azure services, local development, external applications.
Types	Managed Identity: System-assigned (tied to resource lifecycle) or User-assigned (reusable across resources).

IRSA — IAM Roles for Service Accounts (AWS EKS)

In EKS, the equivalent of Managed Identity is IRSA. A Kubernetes ServiceAccount is annotated with an IAM role ARN. An OIDC provider validates the pod's identity and AWS STS issues temporary credentials. The pod can then access AWS services (S3, DynamoDB, etc.) without any static credentials in the code or environment.

10.7 Observability — Metrics, Logs, Traces

The three pillars of observability tell you different things about your system:

Pillar	What It Tells You
Metrics (Prometheus/Grafana)	Numerical data over time. CPU usage, memory, request count, error rate, latency percentiles. Good for dashboards and alerting on thresholds.
Logs (ELK / Azure Monitor)	Detailed event records. What exactly happened, when, with what input. Good for debugging specific incidents after the fact.
Traces (Jaeger / Application Insights)	Follow a single request across multiple microservices. Shows which service is adding latency in a complex call chain.

Prometheus scrapes metrics from pods via HTTP (the /metrics endpoint). Pods expose their metrics using client libraries (Prometheus client for Python, Go, Java). Grafana visualises Prometheus data as dashboards. AlertManager (part of the Prometheus ecosystem) sends alerts to Slack, PagerDuty, email when thresholds are exceeded.

```
# Useful Kubernetes monitoring commands
kubectl top nodes                    # Node CPU and memory
kubectl top pods -n production       # Pod CPU and memory
kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods  # Raw metrics API

# Check Prometheus targets (from inside cluster)
kubectl port-forward svc/prometheus 9090:9090 -n monitoring
# Then browse http://localhost:9090/targets
```

10.8 Interview Questions with Full Answers

Q: Design a secure, scalable architecture for a production application on Azure.

Traffic enters through Application Gateway with WAF enabled, handling TLS termination and layer 7 routing. The Application Gateway is integrated with AKS via AGIC. AKS hosts microservices with HPA for pod scaling and Cluster Autoscaler for node scaling. Microservices access Azure SQL and Blob Storage via Private Endpoints — no public exposure. Secrets and certificates are stored in Key Vault, accessed via Managed Identity with RBAC. Outbound traffic uses NAT Gateway for a fixed IP. Everything is monitored via Prometheus + Grafana + Azure Monitor. Infrastructure is managed as code with Terraform and separate state files per environment.

Q: How do you debug a 502 error from Application Gateway?

A 502 from App Gateway means the gateway received an invalid response from or could not connect to the backend. Step 1: Check the App Gateway backend health in Azure Portal — green means the probe is passing. Step 2: If backend is unhealthy, kubectl describe pod to check if pods are running and readiness probes are passing. Step 3: Check if the health probe path in App Gateway matches an actual endpoint in your application (e.g., probe hitting / but app needs /health). Step 4: Check NSG rules between the App Gateway subnet and the AKS subnet — port 65200-65535 must be open for App Gateway management traffic. Step 5: Check AGIC logs for sync errors.

Days 12 & 13 — Advanced Kubernetes, System Design & Architecture

Goal: Master advanced Kubernetes scheduling, networking internals, and be able to design complete production systems from scratch in an interview.

12.1 Advanced Scheduling — Topology Spread Constraints

Topology Spread Constraints are the modern way to spread pods across failure domains (nodes, zones). Unlike basic anti-affinity, they give you fine-grained control over how evenly pods are distributed.

```
topologySpreadConstraints:
- maxSkew: 1                          # Max difference in pod count between zones
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule   # Hard requirement
  labelSelector:
    matchLabels:
      app: my-api

# This ensures pods are spread as evenly as possible across availability zones.
# maxSkew: 1 means no zone can have more than 1 extra pod vs the least-loaded zone.
```

12.2 Priority Classes and Pod Disruption Budgets

PriorityClasses allow critical workloads to preempt lower-priority pods when cluster resources are full. Kubernetes will evict low-priority pods to make room for a high-priority pod that cannot be scheduled.

PodDisruptionBudgets (PDBs) protect applications from having too many pods evicted simultaneously during voluntary disruptions (node drain, cluster upgrades). They define the minimum number of pods that must remain available.

```
# PodDisruptionBudget: always keep at least 2 pods running
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-api-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: my-api

# Without PDB: kubectl drain node could evict all 3 replicas at once = outage
# With PDB: drain evicts one pod, waits for it to reschedule, then evicts the next
```

12.3 Horizontal Pod Autoscaler (HPA)

HPA automatically scales the number of pod replicas based on observed metrics. It queries the Metrics API every 15 seconds (default) and adjusts replicas to keep the target metric at the desired value.

```
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
metadata:
  name: my-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-api
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70   # Scale up when avg CPU > 70%
  - type: Resource
    resource:
      name: memory
      target:
        type: AverageValue
        averageValue: 400Mi
```

HPA requires resource requests

HPA's CPU utilization metric is calculated as (current CPU) / (requested CPU). If you do not set resource requests on your pods, HPA cannot calculate utilization and will not scale. Always set CPU requests on pods you intend to autoscale.

12.4 Cluster Autoscaler

The Cluster Autoscaler (CA) adds or removes nodes from the AKS cluster based on pod scheduling needs. When a pod cannot be scheduled because there is no node with sufficient resources, CA provisions a new node. When nodes are consistently underutilized, CA drains and removes them.

CA works with AKS node pools. In AKS, configure min and max node counts per node pool, and CA operates within those bounds.

HPA + CA together

HPA scales pods. If pods cannot schedule, CA adds nodes. This combination handles traffic spikes automatically:
Traffic spike → CPU rises → HPA creates more pods → pods cannot schedule (no capacity) → CA adds nodes → pods schedule → system handles load.
Scale-down is the reverse — traffic drops, HPA removes pods, nodes become underutilized, CA removes nodes.

12.5 System Design Framework for Interviews

When asked to design a system, never jump straight to listing tools. Use this framework:

59. Clarify requirements: Scale (how many users, requests/sec), availability (99.9%? 99.99%?), consistency vs availability tradeoffs, compliance requirements, geographic distribution.

60. High-level architecture: Draw the major components and data flow. Frontend → API → Database. Show the network boundaries.
61. Component breakdown: Explain each component's role, technology choice, and why that choice.
62. Data flow: Walk through the complete request lifecycle from user to database and back.
63. Security: Authentication, authorization, encryption in transit and at rest, network segmentation.
64. Scaling: How does each layer scale? Horizontal vs vertical. What are the bottlenecks?
65. Failure handling: What happens when each component fails? Single points of failure? Multi-zone/multi-region?

12.6 Your Gold Answer — Architecture Based on Your Experience

Complete Architecture Answer

Traffic enters through Azure Application Gateway with WAF enabled for protection against OWASP Top 10 vulnerabilities. DNS resolves to the Application Gateway public IP. Application Gateway terminates TLS using certificates from Azure Key Vault.

AGIC translates Kubernetes Ingress rules to Application Gateway backend pool configuration. AKS hosts microservices across multiple availability zones for HA. Services communicate internally via ClusterIP services. HPA scales pods based on CPU/memory metrics. Cluster Autoscaler handles node scaling.

Microservices access Azure SQL, Redis Cache, and Blob Storage via Private Endpoints — no public traffic. Secrets and TLS certificates are stored in Key Vault, accessed via Workload Identity (Managed Identity). AKS pods assume Azure identities via OIDC federation.

All outbound traffic from AKS goes through a NAT Gateway for a deterministic egress IP. NSGs enforce micro-segmentation between subnets. Network Policies enforce pod-level traffic controls.

Observability: Prometheus scrapes pod metrics, Grafana provides dashboards, AlertManager sends alerts. All logs go to Azure Log Analytics via the AMA agent. Application Insights provides distributed tracing.

Infrastructure is managed with Terraform, separate state per environment. CI/CD via Azure DevOps: build → test → push image to ACR → deploy to AKS via Helm or kubectl.

12.7 High Availability Patterns

Pattern	Implementation
Multi-zone node pools	Deploy AKS node pools across 3 availability zones. Pods spread across zones via topology spread constraints.
Multi-replica deployments	Minimum 2 replicas for every production workload. Never run single-replica production services.
Pod Disruption Budgets	Protect from voluntary disruptions during maintenance and upgrades.
Rolling updates	Never bring down the entire application — rolling updates maintain availability.
Health probes	Readiness probes ensure traffic only goes to healthy pods. Liveness probes auto-restart stuck pods.
Circuit breaker	Service mesh (Istio) or application-level circuit breakers prevent cascade failures.

Pattern	Implementation
Multi-region	Active-Active or Active-Passive across Azure regions. Azure Traffic Manager for global load balancing.

12.8 Interview Questions with Full Answers

Q: How would you ensure zero downtime during a production deployment?

Use a Deployment with rolling update strategy. Set maxUnavailable to 0 and maxSurge to 1 — this means no pods are taken down until the new pod is fully ready. Define a readiness probe that only returns healthy when the application is fully initialized. Use PodDisruptionBudgets to protect against accidental mass eviction. Monitor the rollout with kubectl rollout status. If the new version has issues, roll back immediately with kubectl rollout undo. For high-risk changes, use a canary deployment — route 5% of traffic to the new version first and monitor error rates before proceeding.

Q: How do you design for high availability in AKS?

Deploy the AKS cluster with node pools spanning 3 availability zones. Deploy all production workloads with minimum 2 replicas and topology spread constraints to ensure pods are in different zones. Configure HPA and Cluster Autoscaler for dynamic scaling. Use PodDisruptionBudgets to protect against drains during maintenance. Use Azure Application Gateway across zones for HA at the ingress layer. Use Azure SQL with zone-redundant configuration. Deploy Azure Redis Cache with geo-replication for session state. Monitor everything and set alerts on pod crash rates, node capacity, and service latency.

Days 14 & 15 — Mock Interviews, Troubleshooting Scenarios & Final Checklist

Goal: Practice articulating everything you have learned. Work through complex multi-layer debugging scenarios. Identify and fill gaps. Simulate real senior DevOps interview conditions.

14.1 Mega Troubleshooting Scenarios

Scenario: Application returning 503 errors in production

This is a layered debugging problem. Work from outside in:

66. Check Application Gateway: Is the backend health status green? If red, the gateway cannot reach your pods.
67. kubectl get pods -n production — are all pods Running? Any in CrashLoopBackOff or Pending?
68. kubectl describe pod <name> — look at Events, readiness probe status, restart count.
69. kubectl logs <pod> --previous — if pods are restarting, get logs from the crashed instance.
70. kubectl get endpoints <service-name> — are there endpoints? If empty, pods are not matching the service selector or are failing readiness checks.
71. Check HPA: kubectl get hpa — is autoscaling triggered? Is it unable to scale (maxReplicas hit)?
72. kubectl top nodes — is the cluster running out of resources?
73. Check Prometheus/Grafana dashboards for latency spikes, error rate trends, database connection exhaustion.

Scenario: Pods scheduled but not starting, status Pending

74. kubectl describe pod — read Events at the bottom. The Scheduler will explain why it cannot place the pod.
75. Check nodes: kubectl get nodes — any NotReady? Cordon'd?
76. Resource pressure: kubectl describe node — check 'Allocatable' vs 'Allocated resources'. Is CPU or memory exhausted?
77. Taint mismatch: kubectl get nodes -o json | grep taints — does your pod have matching tolerations?
78. PVC issue: kubectl get pvc — is the PVC in Pending state? StorageClass unable to provision?
79. Image pull: Is imagePullBackOff the reason? Check imagePullSecret, registry credentials.

Scenario: Memory leak causing cluster instability

80. kubectl top pods — identify which pod is consuming excessive memory.
81. kubectl describe pod <name> — check if OOMKilled appears in Last State.
82. Grafana: memory usage trend over time — is it growing continuously (leak) or spiking (burst)?
83. If confirmed leak: increase memory limit as temporary relief. Long-term: profile application with heap dump analysis, identify leak source in code.
84. If burst: tune memory limit to match actual peak. Add alerts at 80% threshold.

14.2 Architecture Design Interview Practice

Practice answering each of these within 5 minutes — structure your answer with the framework from Day 12-13:

- Design a CI/CD pipeline for a microservices application with 15 services.
- Design a disaster recovery architecture for an AKS workload.
- Design a secure multi-tenant Kubernetes cluster.
- How would you migrate a monolith to microservices on Kubernetes?
- Design an observability stack for a production AKS cluster.

14.3 Interview Answer Quality Checklist

For each technical question, check your answer against these dimensions:

- Did you explain the WHY, not just the what? Interviewers probe deeper when you only list tools.
- Did you give a real production example from your experience?
- Did you mention failure scenarios and how you handle them?
- Did you mention security implications?
- Was your answer concise? Senior engineers communicate clearly and avoid rambling.

14.4 Quick Reference — Critical kubectl Commands

```
# Cluster overview
kubectl get nodes -o wide
kubectl cluster-info

# Pod debugging
kubectl get pods -A                          # All namespaces
kubectl describe pod <name> -n <ns>
kubectl logs <pod> -c <container> -n <ns>
kubectl logs <pod> --previous                # Last crashed instance
kubectl exec -it <pod> -- bash

# Deployments
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>

# Networking
kubectl get svc -A
kubectl get endpoints <svc>
kubectl exec -it <pod> -- nslookup <service>
kubectl exec -it <pod> -- curl http://<service>:<port>

# Resources
kubectl top nodes
kubectl top pods -n <ns>

# RBAC debugging
kubectl auth can-i get pods --as <user> -n <ns>

# Events (sorted by time)
kubectl get events -n <ns> --sort-by=.lastTimestamp
```

14.5 Final Interview Checklist — Must Be Able To

- Draw Kubernetes architecture on a whiteboard — control plane, worker nodes, components, flows.
- Explain the complete kubectl apply flow naming every component involved.
- Debug any pod failure: Pending, CrashLoopBackOff, OOMKilled, ImagePullBackOff.
- Design a production CI/CD pipeline with security, artifact management, and environment promotion.
- Explain your company's architecture and your specific contributions and decisions.
- Write a Dockerfile from scratch with multi-stage build, non-root user, layer caching.
- Design Terraform modules and explain state management strategy.
- Configure RBAC for a team with different access levels.
- Explain TLS handshake and debug certificate failures.
- Design a complete Azure architecture: App Gateway + AKS + Private Endpoints + Key Vault.
- Explain at least 3 real incidents you resolved — root cause, fix, prevention.
- Answer 'why' questions for every technology choice on your resume.

Interview Success Formula

40% — Deep technical knowledge (this guide gives you this)
30% — Clear communication and structured problem-solving approach
20% — Real project experience and lessons learned
10% — Cultural fit and collaboration examples
Master all four to stand out as a senior engineer.

Good luck with your interviews!

You have the knowledge — now practise explaining it clearly.

Quick Reference — Key Comparisons

Kubernetes Workload Types

Type	Use Case	Key Feature
Deployment	Stateless apps (APIs, web servers)	Rolling updates, rollback, self-healing
StatefulSet	Databases, message queues	Stable identity, stable storage, ordered scaling
DaemonSet	Node agents (logging, monitoring)	One pod per node, auto-adds to new nodes
Job	One-off batch tasks	Runs to completion, retry on failure
CronJob	Scheduled batch tasks	Cron syntax, creates Jobs on schedule

Kubernetes Service Types

Type	Accessible From	Use Case
ClusterIP	Inside cluster only	Microservice-to-microservice communication
NodePort	External via node IP:port	Development, testing — not recommended for prod
LoadBalancer	External via Azure LB public IP	Production internet-facing services
ExternalName	Inside cluster — maps to external DNS	Giving external services a cluster-internal DNS name

Kubernetes Probe Types

Probe	Failure Action	Use For
Liveness	Container killed and restarted	Detecting deadlocks, unrecoverable stuck processes
Readiness	Pod removed from service endpoints	Slow startup, temporary unavailability, cache warming
Startup	Liveness/readiness suspended until passes	Applications needing >30s to initialize

CI/CD Tools Comparison

Tool	Strengths	Best For
Jenkins	Maximum flexibility, huge plugin ecosystem	Complex pipelines, legacy environments, on-premise

Tool	Strengths	Best For
GitHub Actions	Native GitHub integration, marketplace	GitHub-hosted projects, modern cloud-native apps
Azure DevOps	Deep Azure/AKS integration, enterprise features	Microsoft/Azure environments, enterprise compliance
GitLab CI	Built-in DevSecOps, registry, environments	GitLab-hosted, complete DevOps platform
Terraform Cloud	Remote state, policy as code, collaboration	Large-scale IaC with team collaboration needs

Terraform Concepts Quick Reference

Concept	Quick Explanation
terraform init	Download providers and initialise backend. Run once per project.
terraform plan	Preview changes. Always review before apply. Shows create/update/destroy.
terraform apply	Execute the plan. Modifies real infrastructure.
terraform destroy	Destroy all resources managed by this config. Use -target for specific resource.
terraform import	Bring existing infrastructure under Terraform management.
Remote state	Store tfstate in Azure Blob Storage for team access and state locking.
Module	Reusable collection of resources. Call with module {} block, pass variables.
Data source	Read existing infrastructure not managed by this config (data {}).